

DevSecOps 1-2-SDLC

Task 1 - Introduction

This room focuses on the Software Development Lifecycle (SDLC) framework, its phases, and components.

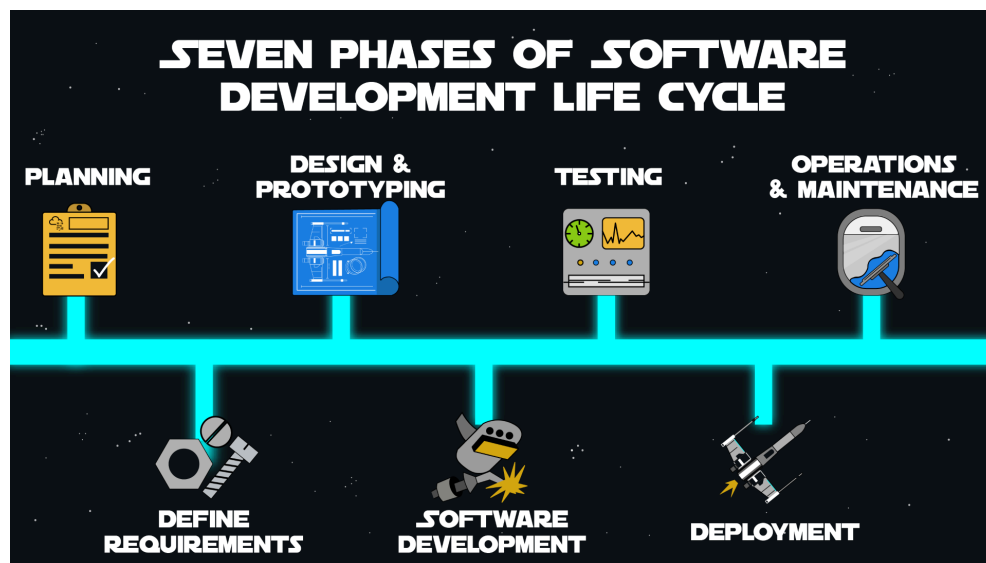
Task 2 - SDLC

What is Software Development Lifecycle (SDLC)?

A standardized framework of practices used to build high-quality software efficiently while meeting cost and time goals.

How does SDLC work?

The aim of SDLC is to establish repeatable processes and predictable outcomes from which future projects can benefit. SDLC phases are usually divided into 6-8 phases. The set of phases are:



- **Planning:** The planning phase encompasses all aspects of project and product management. This includes resource allocation, project scheduling, cost estimation etc.
- **Requirements Definition:** is considered part of planning to determine what the application is supposed to do and its requirements. For example, a social

media application would require the ability to connect with a friend.

- **Design & Prototyping:** establishing how the software application will work, programming language, methods to communicate with each other, architecture etc
- **Software Development:** entails building the program, writing code and documentation.
- **Testing:** In this phase, we ensure components work and can interact with each other. For example, each function works correctly, different parts of the application work seamlessly together, and performance-wise, there are no lags in processing etc.
- **Deployment:** in this stage, the application or project is made available to users.
- **Operations & Maintenance:** this is where engineers respond to issues in the application or bugs and flaws reported by users and sometimes plan additional features in future releases.

Answer the questions below

| How many phases can an SDLC have? (Format X-Y)

6-8

Task 3 - SDLC Phases Part 1

Introduction: The first phases focus on breaking down the project or application before moving on to the development of the application.

1. Planning: Defining project scope, objectives, and resource requirements to establish a foundational roadmap.

2. Requirements Definition: A system's requirements stage involves defining the prototype ideas and gathering all the necessary details. This may be in the form of:

- Making a list of all the requirements for the prototype system
- Prototypes should be evaluated for alternatives
- Identify the end user's needs through research and analysis

3. Design and Prototyping: It is first necessary for developers to outline the details of the overall application, including its attributes, such as:

- Interfaces for users
- Interfaces between systems
- Requirements for networks
- Instances of databases

4. Software Development: Writing and documenting the application's code according to design specifications and organizational standards.

Answer the questions below

| What phase focuses on determining the first idea for a prototype?

Requirements Definition

| What stage is also known as the "Feasibility Stage"?

Planning Stage

| When do you outline the user interfaces and network requirements?

Design and Prototyping

Task 4 - SDLC Phases Part 2

Introduction: The final phases focus on testing the application before release and maintaining its healthy operability afterwards.

5. Testing: Evaluating and verifying software quality and security through structured testing procedures before release.

Test case design and development: Creating detailed, repeatable, and maintainable test cases to validate software requirements and core functionality.

Test environment setup: Configuring the necessary hardware, software, network, and data parameters to replicate real-world user conditions for testing.

Test execution: Running test cases through manual and automated methods to identify bugs, verify performance, and ensure new code fixes do not introduce regressions.

6. Deployment: Releasing the tested software to end-users, often using automation tools for seamless rollouts and rollbacks.

7. Operations and Maintenance: Resolving post-launch user issues, applying necessary updates, and maintaining system stability and uptime. When it comes to the specific part of operations in DevOps, this often means:

- **Self-Service Enablement:** Providing developers with on-demand access to secure, compliant tools and environments to maximize velocity and autonomy.
- **Process Standardization:** Unifying tooling and techniques across the business to reduce friction and improve collaboration between development and operations.
- **Extensible Automation:** Automating repeatable operational tasks—like incident resolution, system updates, and infrastructure scaling—to drive consistency and enable metrics tracking.

Answer the questions below

| What phase focuses on handling issues or bugs reported by end-users?

Operations and Maintenance

| What phase involves releasing new versions of software?

Deployment

| What phase ensures software meets the standards defined in the requirements phase?

Testing

Task 5 - Keep CALMS

Introduction: CALMS, as explained in the Atlassian post, is a framework that assesses a company's ability to adopt DevOps processes. The acronym was coined by Jez Humble, co-author of "The DevOps Handbook," which stands for Culture, Automation, Lean, Measurement, and Sharing.

Culture: Shifting from slow waterfall releases to collaborative, sprint-based workflows across all teams including development, QA, product management, and

operations.

Automation: Implementing reliable, repeatable processes—such as continuous delivery, continuous integration, and configuration-as-code—to streamline component integration and production builds.

Lean: Breaking projects into granular tasks to deliver early functional versions quickly, gathering user feedback for continuous improvement rather than waiting years for a finished product.

Measurement: Tracking effectiveness metrics to evaluate processes and drive continuous improvement through data-informed adjustments.

Sharing: Fostering shared responsibility across development and operations teams for the overall lifecycle and delivery of the final software.

Answer the questions below

| What does CALMS stand for?

Culture, Automation, Lean, Measurement, Sharing

Task 6 - DevOps Metrics

Introduction: It is crucial to understand what metrics DevOps engineers gather before you can advise and introduce security.

DevOps Metrics: Key operational metrics used to track release speed, deployment frequency, and system reliability for continuous improvement. Here are the essential metrics gathered by engineers and the questions it aims to answer:

- Meantime to production ([MTTP\(opens in new tab\)](#)). What is the turnaround time for newly committed source code?
- Frequency of deployment. What is the frequency of deployment of releases? The average lead time. How long does it take to develop, build, test, and deploy a new feature?
- Speed of deployment. A new release is deployed into production; how long does it take?

- **Deployment Agility.** You can measure deployment agility by combining deployment speed and frequency.
- **Production failure rate.** How often do failures occur in production?
- **MTTR** stands for mean time to recover. What is the recovery time after a failure?

MTTP: Measures the time from code commitment to deployment, improved by test automation and small batches.

Failure Rate: Tracks the percentage of production code changes requiring hotfixes or remediation.

Deployment frequency: Evaluates how often new code changes are successfully released into production.

MTTR: Measures the average time required to recover from partial or total service interruptions.

Communicating Risk: Aligns security, development, and operational perspectives on risk to build cross-functional buy-in.

Answer the questions below

| What 2 metrics are used to measure deployment agility?

Deployment speed and frequency

| What is an essential rate for engineers in Production environments to know if code meets security requirements?

Failure rate

| What is the measurement for recovery time after a failure?

MTTR

Task 7 - Production of the Droids

The Empire Expects Great SDLC

Objective: Double the \$1,000,000 seed investment by managing a droid production factory using SDLC principles.

Key Constraint: Apply Brooks's Law—adding human resources to a late software project makes it later, so carefully balance your developer count to maintain sprint efficiency.

Execution: Hire developers, set the number of sprints, and strategically allocate them across the SDLC phases before launching production.

Answer the questions below

What is the flag that you receive once you have doubled the empire's investment?

THM{Ruler.of.the.SDLC.Droids}

Select **6 Developers** and **20 Sprints**.

Strategic Breakdown

- **Avoid Brooks's Law:** The prompt warns that "adding human resources to a late software project will make it later" (Brooks's Law). Keeping the team at **6 developers** prevents efficacy loss from over-staffing overhead while keeping project costs manageable.

- **Maximize Sprint Allocation:** Selecting **20 sprints** gives you the necessary coverage across the SDLC phases without exhausting the seed budget on developer salaries alone.

- **Cost Calculation:** $\$6 \text{ developers} \times 20 \text{ sprints} \times \$3,000 = \$360,000$

This leaves **\$640,000** in remaining budget to handle droid manufacturing and operational overhead during production.

That leaves **\$640,000** from the initial budget. A published walkthrough recommends this moderate team size and sprint count as a strong starting point, avoiding the productivity penalty associated with excessively large teams.



SDLC Factory Game

Select amount of sprints and developers.



Total Budget:

\$1,000,000



Cost of developer per sprint :

\$3,000



Number of developers :

-10

-5

-1

6

+1

+5

+10



Number of sprints :

-10

-5

-1

20

+1

+5

+10



Budget Remaining :

\$640,000

Next

SDLC Phase / Sprints (Distribute all **20 sprints** across the phases exactly as follows):

Planning 1

Define Requirements 5

Design and Prototyping 5

Software Development 5

Testing 2

Deployment 1

Operations and Maintenance 1

- **10 Developers** vastly increases the **Droid Production Rate** (manufacturing speed), which generates significantly more total revenue over time.
- Allocating **5 sprints each** to Requirements, Design, and Development maintains an optimal balance that keeps **Rate of New Bugs** at **0%** and **Defective Droids** at **0%**, ensuring max profit without losing money on fixes.

DevSecOps 1-2-SDLC

8

Once **Remaining Sprints** reaches **0** and no red error boxes remain, click **Start Game** to run the simulation and receive your flag and view your flag
(`THM{Ruler.of.the.SDLC.Droids}`).